

Operating Systems and System Programming

Practical - 1

Title: Execution of Linux Commands Using System Calls (fork(), execvp(), wait())

Aim: To develop a C program that accepts a Linux command from the user, creates a child process using fork(), executes the command using execvp(), waits for the child process to complete using wait(), and displays the Process IDs (PIDs) of both the parent and child processes.

Algorithm:

1. Start the program.
2. Read a Linux command from the user.
3. Split the command into the command and its arguments.
4. Call fork() to create a child process.
5. If it is the child process:
 - Display the Child Process ID (PID).
 - Execute the command using execvp().
6. If it is the parent process:
 - Display the Parent Process ID (PID).
 - Wait for the child process to finish using wait().
7. Print the completion message.
8. End the program.

Process:

Step-1: nano task-1.c

Create a file



```
allur@AKSHAYA:~$ nano task-1.c
```

Step-2: Write the code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```

#include <sys/wait.h>

#include <string.h>

int main() {

    char command[100];

    printf("Enter a Linux command: ");

    fgets(command, sizeof(command), stdin);

    // Remove newline character

    command[strcspn(command, "\n")] = '\0';

    pid_t pid = fork();

    if (pid < 0) {

        printf("Fork failed!\n");

        return 1;

    }

    else if (pid == 0) {

        // Child process

        printf("\nChild Process\n");

        printf("Child PID : %d\n", getpid());

        printf("Parent PID: %d\n", getppid());

        execlp(command, command, NULL);

        // Executes only if execlp fails

        perror("Execution failed");

        exit(1);

    }

    else {

        // Parent process

        printf("\nParent Process\n");

        printf("Parent PID : %d\n", getpid());

        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }
}

```

```
}  
    return 0;  
}
```

Step-3: press ctrl+o

Save the contents of the file.

Step-4: Press Enter

It confirms the filename and completes the save.

Step-5: Press ctrl+x

Exists the nano editor.

Step-6: gcc task-1.c -o task-1

Compiles the c source code into an executable program named task-1

A terminal window with a black background and green text. The prompt is 'allur@AKSHAYA:~\$'. The command 'gcc task-1.c -o task-1' has been entered and executed. The prompt is now 'allur@AKSHAYA:~\$' followed by a vertical cursor bar.

```
allur@AKSHAYA:~$ gcc task-1.c -o task-1  
allur@AKSHAYA:~$ |
```

Step-7: ./task-1

Runs the compiled executable from the current directory

A terminal window with a black background and green text. The prompt is 'allur@AKSHAYA:~\$'. The command './process' has been entered and executed. The prompt is now 'allur@AKSHAYA:~\$' followed by a vertical cursor bar.

```
allur@AKSHAYA:~$ gcc task-1.c -o task-1  
allur@AKSHAYA:~$ ./process  
Enter a linux command: |
```

Step-8: Enter command: date

The date command in an O.S displays the current system date and time.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: date

Parent Process
Parent PID : 787
Child PID  : 788

Child Process
Child PID : 788
Parent PID: 787
Sat Aug  1 09:34:57 UTC 2026

Child process completed.
allur@AKSHAYA:~$ |
```

Step-9: Enter command: uname

The uname command displays information about the operating system and system kernel, such as its name, version, and architecture.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: uname

Parent Process
Parent PID : 805
Child PID  : 806

Child Process
Child PID : 806
Parent PID: 805
Linux

Child process completed.
allur@AKSHAYA:~$ |
```

Step-10: Enter command: lscpu

The lscpu command displays detailed information about the system's CPU architecture, processors, cores, and threads.

```

allur@AKSHAYA:~$ ./task-1
Enter a Linux command: lscpu

Parent Process
Parent PID : 834
Child PID  : 835

Child Process
Child PID : 835
Parent PID: 834
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          46 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 18
On-line CPU(s) list:   0-17
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) Ultra 5 125H
CPU family:             6
Model:                 170
Thread(s) per core:    2
Core(s) per socket:    9
Socket(s):              1
Stepping:               4
BogoMIPS:               5990.39
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clf
                        lush mmx fxsr sse sse2 ss ht syscall nx pdpe1gb rdtscp lm constant_tsc rep_
                        good nopl xtopology tsc_reliable nonstop_tsc cpuid tsc_known_freq pni pclmu
                        lqdq vmx ssse3 fma cx16 pcid sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline
                        _timer aes xsave avx f16c rdrand hypervisor lahf_lm abm 3dnowprefetch ssbd
                        ibrs ibpb stibp ibrs_enhanced tpr_shadow ept vpid ept_ad fsgsbase tsc_adjus
                        t bmi1 avx2 smep bmi2 erms invpcid rdseed adx smap clflushopt clwb sha_ni x
                        saveopt xsavec xgetbv1 xsaves avx_vnni vnni umip waitpkg gfni vaes vpclmulq
                        dq rdpid movdiri movdir64b fsrm md_clear serialize ibt flush_l1d arch_capab
                        ilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:     Microsoft
Virtualization type:    full
Caches (sum of all):
L1d:                    432 KiB (9 instances)
L1i:                    576 KiB (9 instances)
L2:                     18 MiB (9 instances)
L3:                     18 MiB (1 instance)
NUMA:

```

```

NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):     0-17
Vulnerabilities:
Gather data sampling:   Not affected
Ghostwrite:             Not affected
Indirect target selection: Not affected
Itlb multihit:         Not affected
L1tf:                   Not affected
Mds:                    Not affected
Meltdown:               Not affected
Mmio stale data:        Not affected
Old microcode:          Not affected
Reg file data sampling: Not affected
Retbleed:               Mitigation; Enhanced IBRS
Spec rstack overflow:   Not affected
Spec store bypass:      Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1:             Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:             Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSE-eIBRS SW seq
                        uence; BHI BHI_DIS_S
Srbds:                  Not affected
Tsa:                    Not affected
Tsx async abort:        Not affected
Vmscape:                Not affected

Child process completed.
allur@AKSHAYA:~$

```

Step-11: Enter command:lsblk

The lsblk command displays information about all available block storage devices, such as hard drives, SSDs, and their partitions.

```

Child process completed.
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: lsblk

Parent Process
Parent PID : 882
Child PID  : 883

Child Process
Child PID : 883
Parent PID: 882
NAME MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
sda   8:0    0 356.9M  1 disk
sdb   8:16   0 159.4M  1 disk
sdc   8:32   0     2G   0 disk [SWAP]
sdd   8:48   0     1T   0 disk /mnt/wslg/distro
/

Child process completed.
allur@AKSHAYA:~$ |

```

Step-12: Enter command: ps

The ps command displays information about the currently running processes on the system.

```

allur@AKSHAYA:~$ ./task-1
Enter a Linux command: ps

Parent Process
Parent PID : 897
Child PID  : 898

Child Process
Child PID : 898
Parent PID: 897
  PID TTY          TIME CMD
  404 pts/0        00:00:00 bash
  897 pts/0        00:00:00 task-1
  898 pts/0        00:00:00 ps

Child process completed.
allur@AKSHAYA:~$ |

```

Step-13: Enter command: top

The top command displays a real-time view of running processes and system resource usage, such as CPU and memory.

```
allur@AKSHAYA:~$ ./task-1
Enter a Linux command: top

Parent Process
Child Process
Parent PID : 910
Child PID : 911
Child PID : 911
Parent PID: 910
top - 09:45:04 up 35 min, 1 user, load average: 0.00, 0.02, 0.00
Tasks: 26 total, 1 running, 25 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni,100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7651.2 total, 6934.9 free, 602.5 used, 265.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 7048.6 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24168	15360	11556	S	0.0	0.2	0:00.95	systemd
2	root	20	0	3180	2212	2072	S	0.0	0.0	0:00.01	init-systemd(Ub
7	root	20	0	3180	2048	1940	S	0.0	0.0	0:00.00	init
47	root	19	-1	50400	16720	15412	S	0.0	0.2	0:00.27	systemd-journal
83	systemd+	20	0	22416	14416	11920	S	0.0	0.2	0:00.10	systemd-resolve
91	root	20	0	35448	12324	9148	S	0.0	0.2	0:01.04	systemd-udev
181	root	20	0	2888	1948	1820	S	0.0	0.0	0:00.05	chronyd-starter
182	root	20	0	4472	3092	2844	S	0.0	0.0	0:00.01	cron
183	message+	20	0	8820	5436	4760	S	0.0	0.1	0:00.04	dbus-daemon
190	root	20	0	44096	29808	14668	S	0.0	0.4	0:00.22	networkd-dispat
206	root	20	0	18420	9380	8148	S	0.0	0.1	0:00.13	systemd-logind
237	syslog	20	0	220548	5868	4624	S	0.0	0.1	0:00.12	rsyslogd
250	root	20	0	5492	2732	2548	S	0.0	0.0	0:00.00	agetty
279	_chrony	20	0	23804	11212	7088	S	0.0	0.1	0:00.24	chronyd
291	root	20	0	123460	32716	18104	S	0.0	0.4	0:00.19	unattended-upgr
314	_chrony	20	0	12096	2436	1792	S	0.0	0.0	0:00.01	chronyd
402	root	20	0	3184	1116	980	S	0.0	0.0	0:00.00	SessionLeader
403	root	20	0	3200	1252	1108	S	0.0	0.0	0:00.06	Relay(404)
404	allur	20	0	6268	5540	3864	S	0.0	0.1	0:00.10	bash
405	root	20	0	8648	5528	4684	S	0.0	0.1	0:00.01	login
451	allur	20	0	22508	13416	10972	S	0.0	0.2	0:00.10	systemd
454	allur	20	0	22916	4120	2116	S	0.0	0.1	0:00.00	(sd-pam)
489	allur	20	0	6136	5416	3864	S	0.0	0.1	0:00.02	bash
844	root	20	0	1866032	15380	12556	S	0.0	0.2	0:00.10	wsl-pro-service
910	allur	20	0	2768	1816	1720	S	0.0	0.0	0:00.00	task-1
911	allur	20	0	8332	6164	3948	R	0.0	0.1	0:00.00	top

Conclusion:

Linux abstracts hardware resources by providing a simple interface between applications and physical hardware. It efficiently manages CPU scheduling, memory allocation, storage devices, and I/O operations through system services, allowing multiple programs to run safely and efficiently without directly interacting with the hardware.